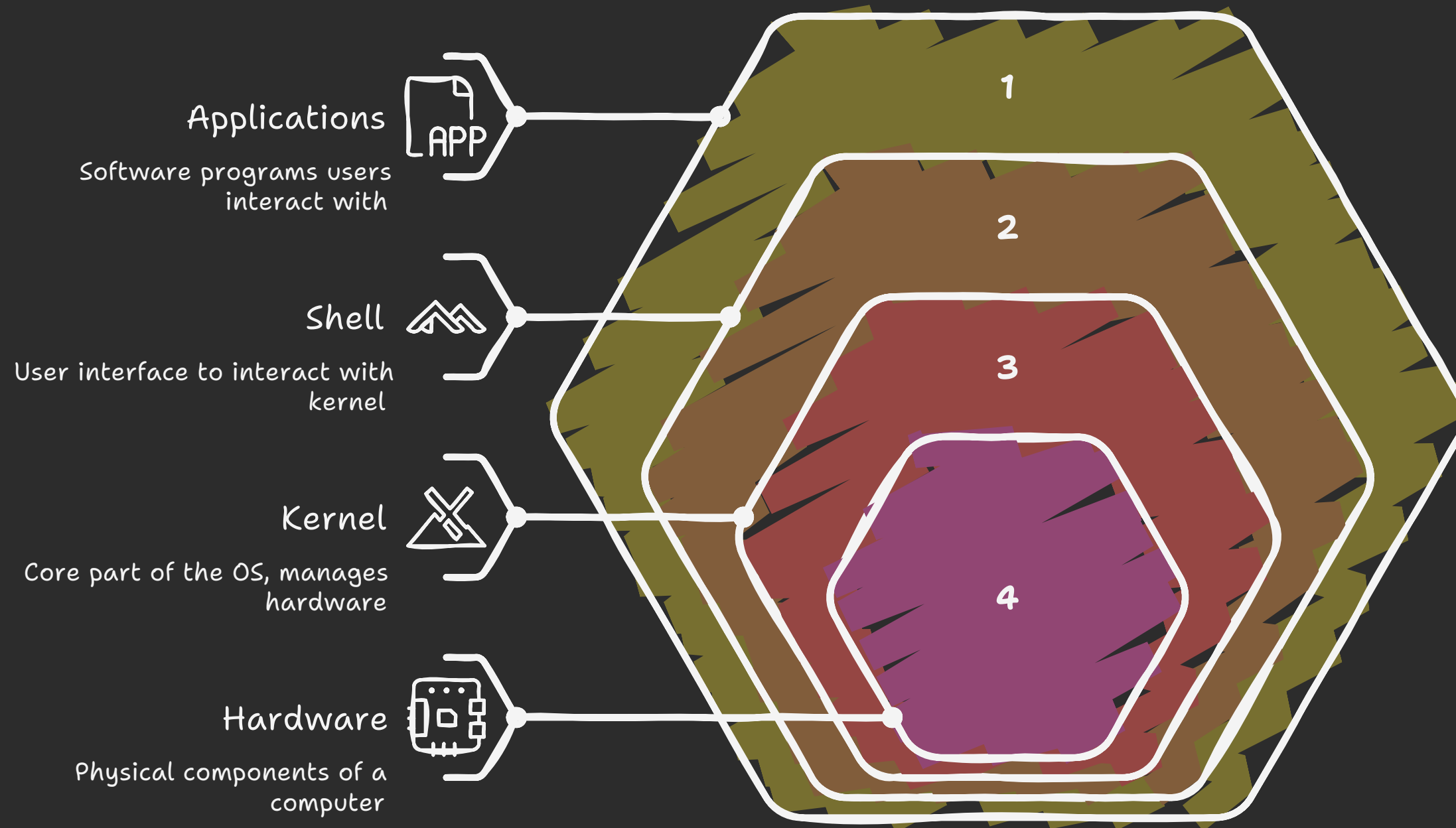


Linux Architecture

Linux Architecture Layers





Hardware

The lowest level of the Linux architecture is the hardware layer. This layer comprises the physical components of a computer, such as the hard drive, RAM, motherboard, CPU, network interfaces, and peripherals. These components are the tangible pieces of your system on which the rest of the architecture is built.



Kernel

Directly interfacing with the hardware layer is the kernel, the heart of the Linux operating system. As the core part of the OS, the kernel is responsible for low-level tasks such as disk management, task scheduling, memory management, and controlling peripherals.

The Linux kernel is a monolithic kernel, meaning it encompasses device drivers, file systems, system server calls, and more, all in a single static binary file. Because the kernel directly interacts with the system's hardware, it's crucial in terms of system performance and stability.



Shell

One layer up from the kernel is the shell. In simplest terms, the shell is a user interface that allows users to interact with the kernel. In Linux, most interactions with the shell occur in a command-line interface [CLI], where users type commands interpreted by the shell.

There are several different shells available in Linux, each with its unique features and syntax, such as the Bourne Again Shell [bash], the C Shell [csh], and the Z Shell [zsh].



Applications

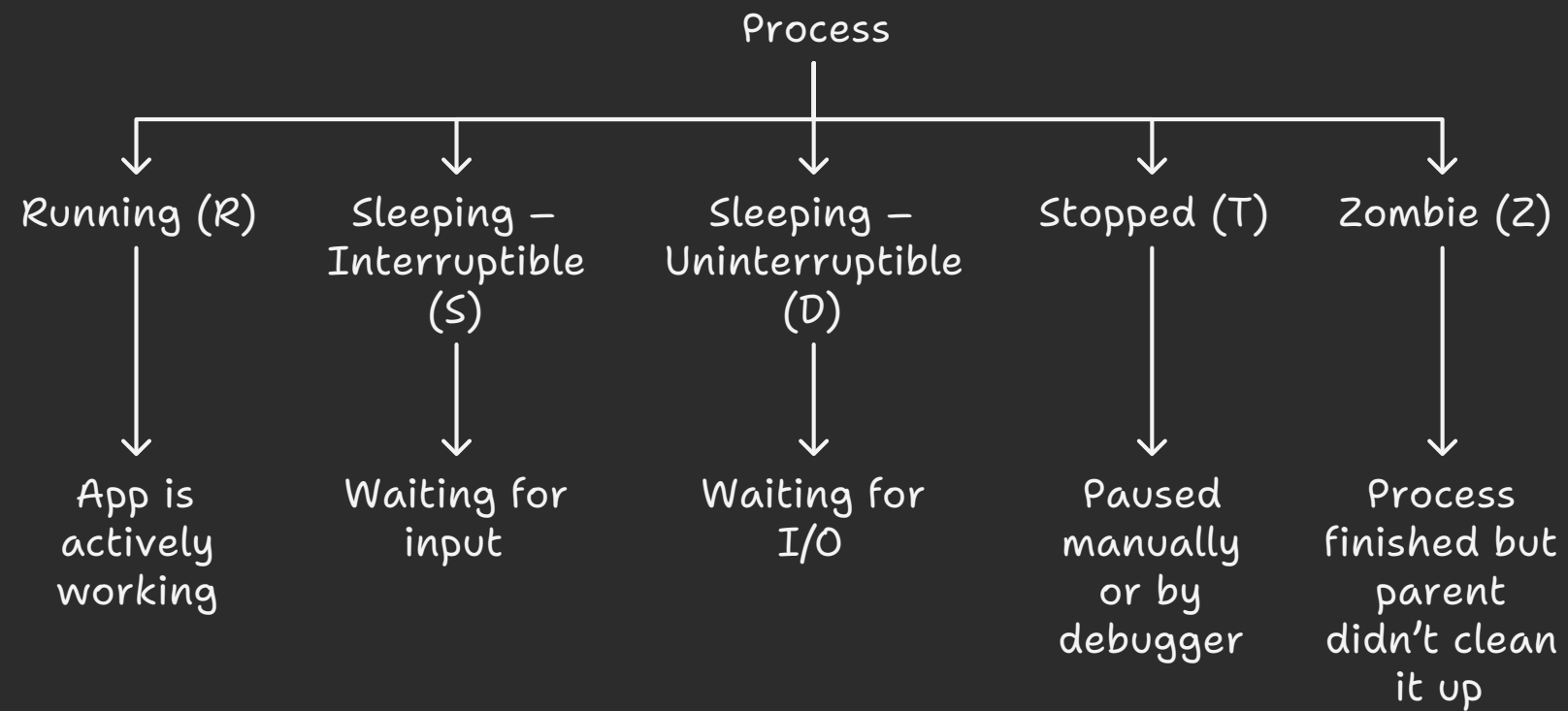
The topmost layer of the Linux architecture consists of applications. These are the software programs that you, as the user, interact with directly. They range from system applications like file managers, text editors, and network managers, to user applications like browsers. While applications communicate with the hardware through the kernel, they interact with the user through the shell. For instance, when you run a command to open a file in a text editor, the shell interprets your command, the kernel fetches the file from the hardware, and the text editor [application] displays it.



Linux Process States (What you actually see & use)

1. **Running [R]** Process is currently executing on CPU or ready to run. *Seen when: app is actively working.*
2. **Sleeping – Interruptible [S]** Waiting for input [disk, network, keyboard]. *Most normal state for background programs.*
3. **Sleeping – Uninterruptible [D]** Waiting for I/O [usually disk]. Cannot be killed easily. *If stuck long → possible disk or hardware issue.*
4. **Stopped [T]** Paused manually or by debugger [Ctrl + Z]. *Can be resumed later.*
5. **Zombie [Z]** Process finished but parent didn't clean it up. *Doesn't use CPU, but many zombies = bad parent process.*

Linux Process States Flowchart





Linux Commands Used Daily (With Options & Examples)

1

ps – Process Status

5. Used to **view running processes** and their details.

Common Options

- a → show processes of **all users**
- u → **user-oriented format** [CPU, memory, user]
- x → show processes **not attached to terminal**

Most Used Command

```
ps aux
```

Useful Commands:

```
ps -ef          # Full-format listing
ps aux | grep ssh # Find specific process
ps -u root      # Processes owned by root
```

Key Columns

- PID – Process ID
- %CPU – CPU usage
- %MEM – Memory usage
- STAT – Process state (R, S, Z, etc.)

2

top – Real-Time Process Monitor

5. Used to **monitor CPU, memory, load, and running processes live.**

Start Command

top

Important Interactive Keys (inside top)

- P → Sort by CPU usage
- M → Sort by memory usage
- k → Kill a process [asks for PID]
- q → Quit top
- 1 → Show CPU usage per core

Alternative (if installed)

```
htop
```

- 5.  More user-friendly, color UI, mouse support

3 kill – Send Signals to Processes

- 5. Used to **stop or control processes using signals**.

Basic Syntax

```
kill <PID>
```

Important Signals

- -15 [SIGTERM] → Graceful stop [default]
- -9 [SIGKILL] → Force kill [cannot be ignored]
- -18 [SIGCONT] → Resume stopped process
- -19 [SIGSTOP] → Pause process

Examples

```
kill 1234  
kill -9 1234  
kill -15 1234
```

Kill by Name

```
pkill firefox  
killall chrome
```

4

df – Disk Free Space

5. Used to **check disk usage of filesystems**.

Most Used Option

```
df -h
```

5. [-h = human readable: GB, MB]

Useful Options

- -T → Show filesystem type
- -i → Show inode usage
- --total → Show total disk usage

Examples

```
df -h  
df -Th  
df -i
```

Why It Matters

- Disk full → services crash
- Logs won't write
- Applications fail silently

5 systemctl – Service Management (systemd)

5. Used to **control and check system services**.

Check Service Status

```
systemctl status nginx
```

Common Options

- start → Start service

- stop → Stop service
- restart → Restart service
- reload → Reload config without downtime
- enable → Start service at boot
- disable → Disable auto-start

Examples

```
systemctl start nginx  
systemctl stop nginx  
systemctl enable ufw  
systemctl disable docker
```